All About Food!

Final Presentation Script

CSS326 Database Programming Laboratory · Group 10 · 3 speakers · about 15 minutes + Q&A

สคริปต์นี้แบ่งเป็น 3 คนตามเกณฑ์ 30 คะแนนของอาจารย์ ทุกช่วงมี 3 กล่อง:

SAY = ประโยคภาษาอังกฤษที่พูด | **ทำบนจอ** = กดอะไร เปิดอะไร | **อธิบาย** = ความหมายและเหตุผลเป็นภาษาไทย

Speaker	Topic	Rubric points	Time
Speaker 1	Overview · ER diagram · Schema	ER & Schema (6)	4 min
Speaker 2	Live demo: open table → order → kitchen → serve → pay · Roles · Menu CRUD · UI	CRUD (7) · Roles (4) · UI (3)	6 min
Speaker 3	Triggers + stored procedure live in Workbench · Security · Tests	SQL features (3) · Security (4)	5 min

ก่อนเริ่ม (5 นาที): เปิด MySQL (`brew services start mysql`) → terminal 1 `cd backend && npm run dev` → terminal 2 `cd frontend && npm run dev` → MySQL Workbench เปิด `restaurant_oms` → เบราว์เซอร์: แท็บลูกค้า `localhost:5173/?table=2` และแท็บพนักงาน login ไว้ (admin, waiter, chef, cashier) → เช็กว่าโต๊ะทุกตัว "ว่าง" ด้วย `SELECT table_id, status FROM dining_table;`

Professionalism 3 คะแนนดูรายคน: อาจารย์อาจถามใครก็ได้เรื่องส่วนไหนก็ได้ ทุกคนควรอ่านทั้งเล่ม โดยเฉพาะหน้า Q&A ท้ายเล่ม ไม่ต้องท่องคำต่อคำ พูดด้วยคำของตัวเองได้ ขอแค่เข้าใจความหมายในกล่องสีเขียว

Speaker 1 — Overview & ER Diagram

About 4 minutes · Rubric: ER Diagram & Database Schema (6 points)

SAY

Good morning, everyone. We are Group 10, and our project is **All About Food!**, a restaurant order management system.

ทำบนจอ

เปิดหน้าเว็บฝั่งลูกค้าค้างไว้บนจอ

อธิบาย (ไทย)

ทักทายและบอกชื่อโปรเจกต์ ใช้เวลาไม่เกิน 10 วินาที

SAY

The problem we wanted to solve is simple. Many small restaurants still take orders on paper. The kitchen can't read the handwriting, orders get lost, and the owner doesn't know the daily sales. With our system, customers scan a QR code at their table and order from their phone. The order goes straight to the kitchen screen, and every staff member only handles their own part of the job.

อธิบาย (ไทย)

อธิบาย "ปัญหา" ก่อน "วิธีแก้" ทำให้อาจารย์เห็นว่าระบบมีเหตุผล ไม่ใช่ทำเพราะต้องส่งงาน ปัญหาคือร้านเล็กจดออเดอร์ด้วยกระดาษ ครัวอ่านไม่ออก ออเดอร์หาย ไม่รู้ยอดขาย ทางแก้คือสแกน QR สั่งเอง ออเดอร์เข้าจอครัวทันที และพนักงานแต่ละตำแหน่งทำเฉพาะงานของตัวเอง

SAY

We built the front end with **React**, the back end with **Node.js and Express**, and the database is **MySQL**.

อธิบาย (ไทย)

บอก tech stack สั้นๆ React = หน้าเว็บ, Node.js + Express = API ฝั่ง server, MySQL = ฐานข้อมูล

SAY

Here is our ER diagram. We have **seven entities** and **six relationships**. I'll explain them in three groups.

First, **people**: the `staff` table stores each employee together with their login account. The role is an ENUM with four values: admin, chef, waiter and cashier.

Second, **the restaurant**: `dining_table` stores each table and its status — available, occupied, or inactive.

Third, **food and orders**: `menu_item`, `ingredient`, `menu_item_ingredient`, `orders` and `order_item`.

ทำบนจอ

เปิด `docs/er-diagram.png` แล้วชี้ไปทีละกลุ่มขณะพูด

อธิบาย (ไทย)

แบ่ง 7 ตารางเป็น 3 กลุ่ม (คน / ร้าน / อาหารและออเดอร์) จะจำง่ายและฟังง่ายกว่าอ่านชื่อทีละตาราง · **ENUM** = คอลัมน์ที่รับได้เฉพาะค่าที่กำหนด MySQL จะไม่ยอมให้ใส่ค่าอื่น · สถานะโต๊ะ: available = ว่าง, occupied = เปิดโต๊ะแล้ว, inactive = ปิดใช้งาน

SAY

We have two **many-to-many** relationships.
The first one is between orders and menu items. One order has many dishes, and one dish appears in many orders, so we use the bridge table `order_item`. It also stores the `subtotal`, so if we change a menu price later, old bills don't change.
The second one is between menu items and ingredients. This is the recipe table, `menu_item_ingredient`. Its primary key is a **composite key**: menu_item_id plus ingredient_id.

ทำบนจอ

ชี้ `order_item` และ `menu_item_ingredient` บน ER

อธิบาย (ไทย)

ส่วนนี้อาจารย์ชอบถามที่สุด **M:N (many-to-many)** ต้องมีตารางกลางเสมอ
• ออเดอร์ ↔ เมนู ใช้ `order_item` และเก็บ subtotal ไว้ เพื่อให้บิลเก่าเป็นราคา ณ ตอนสั่ง
• เมนู ↔ วัตถุดิบ ใช้ `menu_item_ingredient` (สูตรอาหาร) PK เป็น 2 คอลัมน์รวมกัน เรียกว่า **composite key** กันการใส่วัตถุดิบเดิมซ้ำในเมนูเดียวกัน

SAY

The `orders` table has **two foreign keys** to `staff`: `staff_id` is the person who took the order, and `paid_by` is the cashier who received the payment. That is how the cashier's name appears on the receipt.

อธิบาย (ไทย)

จุดที่แสดงว่าเข้าใจ FK: ตารางเดียวกันชี้ไปตารางเดียวกันได้ 2 เส้น ถ้าความหมายต่างกัน (คนรับออเดอร์ vs คนรับเงิน)

SAY

The schema in MySQL matches the ER diagram exactly. Here you can see the foreign keys. A menu item that has already been ordered uses **ON DELETE RESTRICT**, so it cannot be deleted. This protects the sales history.

ทำบนจอ

Workbench: `SHOW CREATE TABLE orders;` แล้วชี้บรรทัด `FOREIGN KEY`

อธิบาย (ไทย)

เกณฑ์ข้อ 1 บอกว่า "schema in MySQL matches ERD" จึงต้องเปิดให้ดูของจริงใน MySQL · **ON DELETE RESTRICT** = ห้ามลบแถวแม่ถ้ายังมีแถวลูกอ้างถึงอยู่

SAY

At first we designed eleven tables, but we removed tables that the app didn't really use and merged a one-to-one pair. Now every table has a clear purpose. Next, Speaker 2 will show you the system live.

อธิบาย (ไทย)

พูดก่อนที่อาจารย์จะถาม ว่าทำไมเหลือ 7 ตาราง: ตัด `table_session` ที่ไม่ได้ใช้ · รวม `employee` + `sys_user` (1:1) เป็น `staff` · ตัดระบบจองโต๊ะที่ไม่ใช่งานหลัก แล้วส่งต่อให้คนที่ 2

Speaker 2 — Live Demo, Roles & CRUD

About 6 minutes · Rubric: CRUD (7) · Access Control & Roles (4) · UI & Navigation (3)

SAY

Let's imagine a customer walks in and sits at table 2. They scan the QR code, choose two plates of basil fried rice, and press "order"...

ทำบนจอ

แท็บลูกค้า `/?table=2` → เพิ่ม ข้าวผัดกะเพรา × 2 → ตะกร้า → กดยืนยันสั่ง → ขึ้น "โต๊ะนี้ยังไม่เปิด"

อธิบาย (ไทย)

เริ่มด้วยเคสที่สั่งไม่ได้ก่อน เพื่อโชว์ระบบป้องกัน

SAY

...and it doesn't work. The table is not open yet. This stops people from taking a photo of the QR code and ordering from home.

อธิบาย (ไทย)

อธิบายว่าทำไมระบบปฏิเสธ: กันคนเอาลิงก์ QR ไปสั่งจากนอกร้าน ส่วนนี้คนที่ 3 จะโชว์ trigger ที่ทำงานเบื้องหลังอีกที

SAY

The waiter opens the table from the **Tables** page. Now the customer can order.

ทำบนจอ

แท็บ waiter → เมนู "โต๊ะ" → กด "เปิดโต๊ะ" ที่ T2 → กลับแท็บลูกค้า เลือกวิธีจ่าย QR (ให้เห็นรูปพร้อมเพย์) → กดสั่ง → ไปหน้าติดตามสถานะ

อธิบาย (ไทย)

เปิดโต๊ะ = เปลี่ยน `dining_table.status` จาก available เป็น occupied ซึ่งเป็นการ **Update** ข้อมูลผ่านหน้าเว็บ

SAY

The chef sees the new order right away. The chef can only press "Start cooking" and "Done". There is no cancel or payment button, and no Tables, Menu or Dashboard page.

ทำบนจอ

แท็บ chef → เริ่มทำ → ทำเสร็จ (หน้าลูกค้าอัปเดตเอง)

อธิบาย (ไทย)

นี่คือการโชว์ **Roles**: แต่ละ role เห็นปุ่มไม่เหมือนกัน ให้ชี้ว่า "ไม่มีปุ่ม..." ด้วย เพราะเกณฑ์ต้องการเห็น **restricted actions** (สิ่งที่ทำไม่ได้)

SAY

The waiter marks it as served. The waiter cannot start cooking — the system only allows the order to move forward one step at a time.

ทำบนจอ

แท็บ waiter → กด "เสิร์ฟแล้ว"

อธิบาย (ไทย)

สถานะเดินทีละขั้น: placed → preparing → ready → served → completed ข้ามขั้นหรือย้อนกลับไม่ได้ (backend ตอบ 409)

SAY

Finally, the cashier takes the payment. For cash, the system calculates the change. The receipt shows the cashier's name and 7% VAT.

ทำบนจอ

แท็บ cashier → "ชำระเงิน" → เงินสด รับ 200 → เห็นเงินทอน → ยืนยัน → ใบเสร็จ

อธิบาย (ไทย)

cashier ทำได้อย่างเดียวคือรับเงิน ชื่อบนใบเสร็จมาจาก FK `paid_by` ที่คนที่ 1 พูดถึง ช่วยเชื่อมให้เห็นว่าระบบต่อเนื่องกัน

SAY

All permissions are in one file, `permissions.js`. We have four roles — admin, chef, waiter and cashier — plus customers, who don't need to log in. The real check happens in the **back end**: every request is checked with a JWT token and the user's role. Even if someone calls the API directly, without the website, they get a **403 Forbidden** error. The front end only hides buttons for convenience.

ทำบนจอ

เปิด `backend/src/config/permissions.js`

อธิบาย (ไทย)

ประโยคสำคัญที่สุดของข้อ Roles: **"ความปลอดภัยจริงอยู่ที่ backend"** ถ้าตรวจแค่หน้าเว็บ (ซ่อนปุ่ม) จะเสี่ยงถูกหักคะแนนว่า "hardcoded checks / weak distinction" · **JWT** = token ที่ได้ตอน login เก็บ role ไว้ข้างใน · **403** = login แล้วแต่ไม่มีสิทธิ์

SAY

Now the admin. The admin can do full CRUD on the menu. **Create**: I add "Omelette on rice" for 45 baht. **Read**: it shows up on the customer page. **Update**: I change the price to 50. I can also hide it, so customers can't see it. **Delete**: I remove it.

ทำบนจอ

แท็บ admin → เมนู → เพิ่ม "ข้าวไข่เจียว" 45 → แก้เป็น 50 → ซ่อน → ลบ · ระหว่างนั้นสลับไป Workbench `SELECT * FROM menu_item ORDER BY menu_item_id DESC;`

อธิบาย (ไทย)

พูดคำว่า Create / Read / Update / Delete ออกมาตรงๆ ทีละคำ ให้อาจารย์ติ๊กคะแนนง่าย และต้องสลับไปให้เห็นว่า **ข้อมูลใน MySQL เปลี่ยนทันที** เพราะเกณฑ์เขียนไว้ว่า "reflect instantly in MySQL"

SAY

If I try to delete basil fried rice, which has already been ordered, the system hides it instead. The foreign key protects the old orders. And if I enter a negative price, the system shows an error and saves nothing.

ทำบนจอ

ลองลบ "ข้าวผัดกะเพรา" → ขึ้น "ซ่อนแทนการลบ" → กด "แสดง" คืน · ลองเพิ่มเมนูราคา -5 → ขึ้น error

อธิบาย (ไทย)

โชว์ **validation / error handling** ที่เกณฑ์ CRUD ระบุไว้ และโชว์ว่า FK ทำงานจริง (ต่อจากที่คนที่ 1 พูด)

SAY

For the UI, you can switch between Thai and English, use dark mode, and it works on a phone too. Now Speaker 3 will show what happens inside the database.

ทำบนจอ

กดสลับ ไทย/EN · โหมดมืด (ฝั่งพนักงาน)

อธิบาย (ไทย)

เก็บคะแนน UI สั้นๆ แล้วส่งต่อ

Speaker 3 — Triggers, Stored Procedure & Security

About 5 minutes · Rubric: Advanced SQL Features (3) · Database Security (4)

> **SAY**
> During the demo, several things happened automatically inside the database. We use **three triggers** and **one stored procedure**.

ทำบนจอ

เปิด `database/triggers.sql` สั้นๆ แล้วสลับไป Workbench

อธิบาย (ไทย)

Trigger = โค้ด SQL ที่ทำงานเองเมื่อมี INSERT / UPDATE / DELETE ในตาราง · **Stored procedure** = ชุดคำสั่ง SQL ที่เก็บไว้ใน DB แล้วเรียกใช้ด้วย `CALL`

> **SAY**
> **Trigger number two** checks the table. Table 2 is closed again now, because the bill was paid. Even if I insert an order directly into the database, without the website, the trigger rejects it with a **SIGNAL**.

```
INSERT INTO orders (table_id, order_type) VALUES (2, 'dine_in');
-- Error 1644: Table is not open: please ask staff to open the table first
```

อธิบาย (ไทย)

ชื่อ trigger: `trg_orders_bi_table_open` (BEFORE INSERT on orders) · **SIGNAL** = คำสั่งให้ trigger โยน error เพื่อยกเลิกคำสั่งนั้น · จุดเด่นคือกันได้ถึงระดับฐานข้อมูล ถึงจะไม่ผ่านเว็บก็ตาม

> **SAY**
> **Trigger number one** updates the stock. When we ordered two plates of basil fried rice, the trigger read the recipe from `menu_item_ingredient` and took out 200 grams of minced pork and 2 eggs. If there are not enough ingredients — for example, 99 plates of pad thai — it uses SIGNAL to cancel the whole order.

```
SELECT ingredient_name, quantity_in_stock FROM ingredient WHERE ingredient_name IN ('หมูสับ','ไข่ไก่');
```

ทำบนจอ

ให้คนที่ 2 ลองสั่ง ผัดไทย × 99 บนเว็บ (ต้องเปิดโต๊ะก่อน) → ขึ้น "Out of stock"

อธิบาย (ไทย)

ชื่อ trigger: `trg_order_item_bi_stock` (BEFORE INSERT on order_item) ทำงานทุกครั้งที่เพิ่มรายการอาหาร · trigger นี้ใช้ความสัมพันธ์ M:N ของสูตรอาหาร ถ้าอาจารย์ถามเรื่อง "relationships in queries" ยกตัวอย่างนี้ได้เลย

SAY

Trigger number three runs after an order changes. If an order is cancelled, it puts the ingredients back. When the last bill of a table is paid, it closes the table automatically. That's why table 2 is available again.

```
SELECT table_id, status FROM dining_table;
```

อธิบาย (ไทย)

ชื่อ trigger: `trg_orders_au_status` (AFTER UPDATE on orders) · ยกเลิก = คืนสต็อก แต่โต๊ะยังเปิด ลูกค้าสั่งใหม่ได้ · จ่ายเงินครบ = โต๊ะกลับเป็น available

SAY

Our stored procedure is `sp_sales_summary`. You give it a date, and it returns that day's revenue, number of paid orders, average bill, cancelled orders and dishes sold. The admin dashboard calls this procedure every time.

```
CALL sp_sales_summary(CURDATE());
```

ทำบนจอ

เปิดแดชบอร์ด ชี้ตัวเลขที่ตรงกับผลใน Workbench และข้อความ "คำนวณโดย stored procedure" → เปลี่ยนวันที่ให้ดู

อธิบาย (ไทย)

เกณฑ์ต้องการ "relevant and functional" แปลว่า procedure ต้องถูกใช้จริงในแอป ไม่ใช่สร้างทิ้งไว้ จึงต้องเปิดแดชบอร์ดให้เห็นว่าตัวเลขมาจาก procedure นี้ · โค้ดที่เรียกอยู่ใน `dashboardModel.js` : `CALL sp_sales_summary(?)`

SAY

Now, security. We do four things.
One: every query is **parameterized** with question marks. We never build SQL by joining strings. If I type `admin' OR '1'='1` on the login page, it doesn't work.
Two: no credentials in the code. The database password and the JWT secret are in a `.env` file, which is not uploaded to GitHub.
Three: errors don't leak database details. An unexpected error only returns "Internal server error"; the details stay in the server log.
Four, which is optional: passwords are stored as **bcrypt hashes**, never as plain text.

ทำบนจอ

เปิด `models/orderModel.js` ชี้ `?` · ลอง SQL injection ที่หน้า login · เปิด `.env.example` + `.gitignore` · เปิด `middleware/errorHandler.js` · Workbench `SELECT username, password_hash FROM staff;`

อธิบาย (ไทย)

ตรงกับเกณฑ์ Security ทั้ง 3 ข้อ + แต้มเสริม
• **Parameterized query**: ใส่ `?` แทนค่า แล้วส่งค่าแยก ทำให้ข้อความที่ผู้ใช้พิมพ์ไม่ถูกตีความเป็นคำสั่ง SQL กัน **SQL injection**
• **.env**: ไฟล์เก็บค่าลับ ไม่ commit ขึ้น git
• **Generic error**: ไม่บอกชื่อตารางหรือ SQL ที่พังให้คนนอกรู้
• **bcrypt**: hash ทางเดียว ถอดกลับไม่ได้ ตอน login ใช้ `bcrypt.compare` เทียบ

SAY

Finally, we have **55 automated tests**. They call the real API and check every role, every trigger, the CRUD operations and the stored procedure. They all pass, and the test data is cleaned up afterwards. Thank you. We're happy to answer any questions.

ทำบนจอ

terminal: `cd backend && npm run test:e2e` → **55 passed, 0 failed**

อธิบาย (ไทย)

ปิดท้ายด้วยหลักฐานว่าระบบทำงานจริงทั้งหมด แล้วเชิญถาม

Q&A — likely questions

ทุกคนอ่าน · คอลัมน์ขวาอธิบายเป็นภาษาไทยว่าคำตอบหมายถึงอะไร

Question	Answer (say this)	อธิบาย
Why do you have 7 tables, not 11 like your first design?	We removed a table the app never used, merged a one-to-one pair (employee and login account) into `staff`, and removed the booking feature because it wasn't the main flow.	ตัด table_session · รวม 1:1 · ตัดระบบจอง ทุกตารางที่เหลือถูกใช้จริง
How do you store roles? Why not a separate role table?	It's an ENUM column in `staff`. There are only four roles and each person has one role, so MySQL can enforce the valid values without a join. The permissions for each role are in `permissions.js`.	ENUM บังคับค่าได้ ไม่ต้อง JOIN · สิทธิ์ของแต่ละ role อยู่ในโค้ดไฟล์เดียว
Can someone skip the website and call the API directly?	No. The back end checks the JWT and the role on every request: 401 if not logged in, 403 if the role is not allowed, 409 if they try to skip a step.	401 = ไม่ได้ login · 403 = ไม่มีสิทธิ์ · 409 = ข้ามขั้นหรือสถานะไม่ถูก
Can a customer order from home?	No. They can only order at a table that a staff member has opened. We check it twice — in the back end and in a trigger — and the table closes automatically after payment.	ต้องเปิดโต๊ะก่อน ตรวจ 2 ชั้น จ่ายแล้วโต๊ะปิดเอง
Why do you store `subtotal` in `order_item`? You can calculate it.	Menu prices can change. An old bill must keep the price from the time of the order.	เก็บราคา ณ ตอนสั่ง ไม่ใช่ข้อมูลซ้ำซ้อน
Why can't you delete some menu items?	The foreign key from `order_item` uses ON DELETE RESTRICT to protect the order history, so the system hides the item instead by setting `is_active` to 0.	ลบไม่ได้เพราะ FK · ซ่อนแทน
What if two people press the button at the same time?	Ordering and payment run inside a transaction with `SELECT … FOR UPDATE`, and status updates use `WHERE status = the old status`. If someone changed it first, the second request gets a 409.	transaction + ล็อกแถว กันข้อมูลชนกัน
Why is there no field for card numbers?	Cards are paid on the bank's card machine. Our system only records that the bill was paid by card. We don't store card numbers, for security.	รูดที่เครื่อง EDC ระบบแค่บันทึกวิธีจ่าย
Why check "table open" in both the code and a trigger?	The back end gives a clear error message to the user. The trigger protects the database itself, even if someone inserts data directly.	backend = ข้อความอ่านง่าย · trigger = กันที่ระดับ DB
Why doesn't the procedure run on your online website?	The online version uses TiDB Cloud's free tier, which doesn't support triggers or stored procedures. The back end has a fallback query, and we demo this part on MySQL.	เว็บออนไลน์ใช้ TiDB ฟรี ไม่มี trigger/procedure จึง demo ด้วย MySQL ในเครื่อง

How does password hashing work?	We use bcrypt with cost factor 10. At login we compare with `bcrypt.compare`. We never store or log the real password.	hash ทางเดียว ถอดกลับไม่ได้
How does the app connect to the database?	The settings are in `.env`. `config/db.js` creates a connection pool, the models run the SQL, controllers check the input, and routes connect URLs to controllers.	.env → db.js → models → controllers → routes

Useful phrases & vocabulary

ประโยคช่วยตอนพูดติด หรือตอนตอบไม่ได้

English	ใช้ตอนไหน / ความหมาย
Let me show you on the screen.	ขอโชว์บนจอ (ใช้ตอนอธิบายด้วยคำพูดยาก ให้กดให้ดูแทน)
Could you repeat the question, please?	ขอให้อาจารย์ถามซ้ำ
Good question. In our system, …	ซื้อเวลาคิด 2 วินาทีก่อนตอบ
My teammate can explain this part in more detail.	ส่งต่อให้เพื่อน (ใช้ได้ แต่อย่าใช้บ่อย เพราะคะแนน Professionalism ดูรายคน)
As Speaker 1 mentioned, …	เชื่อมกับที่เพื่อนพูดไปแล้ว ทำให้ดูเป็นทีม
foreign key / primary key / composite key	FK / PK / PK หลายคอลัมน์
many-to-many / one-to-many / bridge table	M:N / 1:N / ตารางกลาง
trigger fires / rejects / signals an error	trigger ทำงาน / ปฏิเสธ / โยน error
parameterized query / SQL injection	query ที่ใช้ ? / การโจมตีด้วยการแทรก SQL
role-based access control	การแบ่งสิทธิ์ตามตำแหน่ง
transaction / commit / roll back	ชุดคำสั่งที่สำเร็จทั้งหมดหรือยกเลิกทั้งหมด

อ่านออกเสียงยาก: *ENUM* = "อี-นัม" · *schema* = "สกี-มา" · *bcrypt* = "บี-คริปต์" · *JWT* = "เจ-ดับเบิลยู-ที" · *SQL* = "เอส-คิว-แอล" หรือ "ซี-เควล" · *ingredient* = "อิน-กรี-เดียนท์"

Detailed click-by-click steps: `docs/demo-test-cases.md` · Thai version of the script: `docs/presentation-script.md`